

ReLU.chat Local-First Optimization Report

A development-only technical audit and roadmap for making ReLU.chat more useful, faster, more intelligent, more conversation-aware, more adaptive in answer length and style, and capable of rendering safe local SVG/HTML diagrams while keeping all user data inside the browser on desktop and mobile.

GENERATED

2026-06-10, Europe/Istanbul

WORKSPACE

/home/yemre/Desktop/FTPSites-personal/relu.chat

DELIVERY

PDF saved under ignored local `dev/`

BOUNDARY

No production code, no deployment, no GitHub push

Bottom line: ReLU.chat already has a strong privacy-first architecture: static hosting, local MiniLM embeddings, BOW fallback, BM25, dense-sparse ranking, MLP policy planning, service-worker caching, and session memory. The largest gains now come from a bot-pack asset format, workerized inference, true indexed retrieval, deterministic composition, local feedback/personalization, diagram AST rendering, and a tighter training and evaluation loop for the policy network.

This file is intentionally development-only. The root `.gitignore` ignores `dev/` and `dev/**`; `git ls-files dev` returned 0 tracked files.

Table Of Contents

1. Audit Evidence
2. Current System Map
3. Priority Roadmap
4. Algorithm Optimizations
5. Conversation Awareness
6. Answer Variation And Length Control
7. Visual SVG/HTML Diagram Layer
8. RL And Policy Pipeline Improvements
9. Asset Loading And Mobile/Desktop Performance
10. Local-Only Data Model
11. Separation Of Concerns And Maintainability
12. Testing, Benchmarks, And Observability
13. Implementation Tracks
14. Completion Checklist

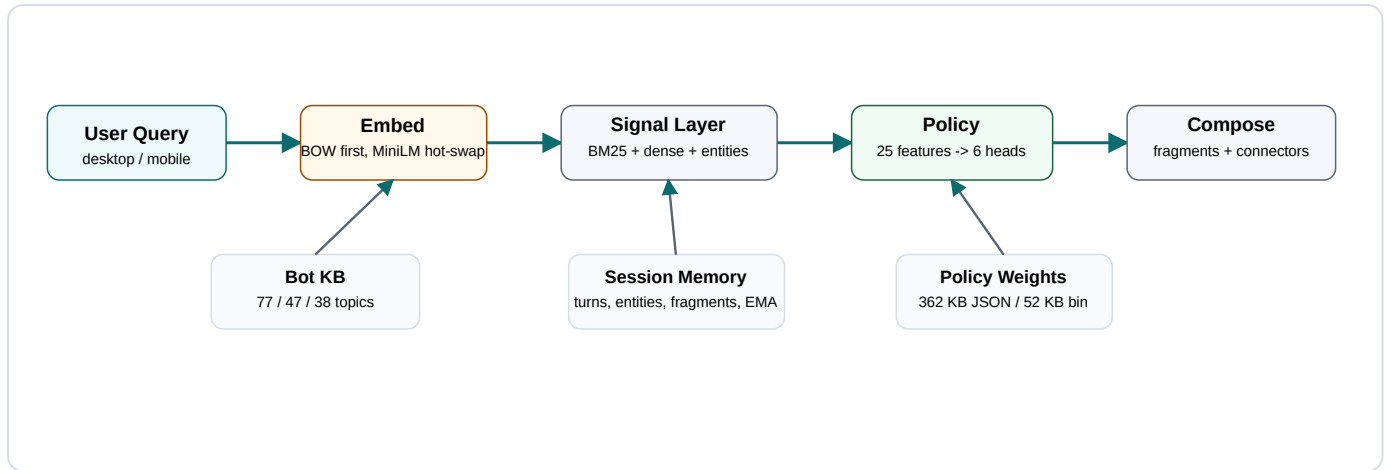
1. Audit Evidence

The report is based on live inspection of the current worktree, source files, development notes, unit tests, dev scripts, asset sizes, and git ignore state. I did not rely on public marketing claims alone.

EVIDENCE	OBSERVED RESULT	WHY IT MATTERS
Git state before report generation	<code>git status --short</code> returned clean before creating dev-only report files.	Establishes that the report work did not mask unrelated source changes.
Git tracking for <code>dev/</code>	<code>git ls-files dev wc -l</code> returned <code>0</code> ; <code>git check-ignore -v dev/...</code> points to root <code>.gitignore:28 dev/</code> .	The PDF can live under <code>dev/</code> without entering GitHub or production by normal git flow.
Relevant runtime/config files	43 source/config/model JSON files under <code>chat/</code> , <code>data/</code> , <code>core/</code> , <code>policy/</code> , and <code>assets/models/policy/</code> .	Shows the engine is compact enough to reorganize into a clearer bot-pack model.
Knowledge base coverage	Game theory: 77 entries / 770 fragments. Golden Age: 47 / 470. Data science: 38 / 380.	Current KB size is small enough that simple algorithms work today, but the next scalable format should be prepared before adding many bots.
Largest cold-start assets	Quantized ONNX model 22.97 MB; four ONNX Runtime WASM variants total about 38 MB; Transformers.js 898 KB; KaTeX 1.5 MB; hero videos about 2.48 MB each.	Cold-start optimization is mostly asset scheduling and model packaging, not policy MLP speed.
Focused unit tests	Cache 50 pass; BM25 34 pass; NLP utils 89 pass; session 53 pass; feature extractor 54 pass; action schema 53 pass. Total focused tests: 333 pass, 0 fail.	Core helper behavior is covered and currently healthy.
Signal layer dev test	<code>node dev/scripts/test-signal-layer.js</code> : 67 pass, 0 fail.	DecisionPacket shape, BM25, ensemble ranking, and follow-up plumbing have working test coverage.
Policy runtime dev test	<code>node dev/scripts/test-policy-runtime.js</code> : 145 pass, 0 fail.	Plan validation, feature packing, heuristic planning, and MLP inference validate in Node.
Quality test	<code>node dev/scripts/test-quality.js</code> : 139 pass, 0 fail across KB, MLP, session, policy, edge, cross-bot.	Broad dev-only harness confirms structural health across all bots.
Follow-up benchmark	<code>node dev/benchmarks/test-followup.js</code> : 97 pass, 3 fail. <code>how?</code> and <code>why?</code> are typed as <code>elaborate</code> in direct/session detection, while feature extractor expects distinct <code>how</code> / <code>why</code> codes.	Concrete conversation-awareness bug: type mapping currently loses nuance for some important one-word follow-ups.
Policy behavior benchmark	Trained MLP on 10 hand-authored scenarios: 70% mode accuracy, 30% intent accuracy. Random baseline: 10%/10%.	The policy improves mode choice but intent training/evaluation needs much better calibration and data.

2. Current System Map

ReLU.chat is a static, browser-only chatbot platform. There is no backend in the conversation path. The browser loads bot-specific KB files and shared engine modules, computes embeddings locally, ranks fragments locally, plans the response with a small local MLP policy, and renders text with local KaTeX.



Strong Current Design Choices

- Static hosting and no server-side inference: the privacy story is simple and defensible.
- Local model files under `assets/models/` with `env.allowRemoteModels = false`.
- Progressive BOW bootstrap lets early turns happen before MiniLM finishes loading.
- BM25 and dense ranking are fused before policy planning.
- The policy contract is explicit through `AnswerPlan` validation.
- Session memory tracks history, recent entities, fragment usage, ambiguity, engagement, and an EMA query vector.
- Service worker already handles model caching and Range requests.

Current Gaps To Treat As Product Opportunities

- Follow-up type drift: `how?` and `why?` are currently collapsed to `elaborate` in one path.
- Seeded composition is only partially wired. `Math.random()` and unseeded `pick()` remain in shared and legacy paths.
- Legacy `chat/*/js/nlp.js` files are still present even though current chat wrappers import `core/chatbot-engine.js`.
- Knowledge, fragment metadata, policy weights, and config are split across several shapes instead of a single bot-pack manifest.
- Policy accuracy is not yet strong enough for high-confidence claims about intent choice.
- Answer length is mostly a side effect of fragment count rather than an explicit budget chosen by policy.
- There is no safe diagram AST, no diagram metadata in fragments, and no renderer contract for visual answers.
- MiniLM and ONNX Runtime loading still dominate cold-start time, especially on mobile.

3. Priority Roadmap

Priorities are ranked by user impact, local-first fit, implementation risk, and how much each unlocks later work.

PRIORITY	WORK	IMPACT	KEY FILES / AREA
P0	Fix follow-up type precedence for <code>how?</code> and <code>why?</code> ; add regression tests.	Directly improves conversation awareness and unlocks different answers for mechanism vs reason questions.	<code>core/session.js</code> , <code>dev/benchmarks/test-followup.js</code> , <code>tests/test-session.js</code>
P0	Create an explicit answer budget in the policy: <code>short</code> , <code>medium</code> , <code>long</code> , <code>auto</code> , and <code>diagram_plus_text</code> .	Meets the requirement for automatic response lengths and variation without user prompting.	<code>policy/action-schema.js</code> , <code>policy/feature-extractor.js</code> , <code>policy/mlp-inference.js</code> , <code>core/nlp.js</code>
P0	Replace remaining unseeded randomness with a deterministic per-turn seed.	Allows reproducible variation, better tests, and stable local personalization.	<code>core/nlp.js</code> , <code>policy/policy-runtime.js</code> , <code>legacy chat/*/js/nlp.js</code>
P1	Introduce <code>bot.pack.json</code> per bot containing KB metadata, fragment vectors, alias index, diagram hints, config, and policy pointers.	Faster loading, fewer requests, cleaner separation of concerns, easier offline caching.	<code>data/bots/*</code> , <code>dev/scripts/*</code> , <code>core/chatbot-engine.js</code>
P1	Move embedding, ranking, and policy planning into a Web Worker with streaming status messages.	Desktop and mobile UI remain responsive while MiniLM and ranking work run off the main thread.	New <code>core/engine-worker.js</code> , <code>core/chatbot-engine.js</code>
P1	Build a safe diagram AST renderer and add diagram candidates to fragment metadata.	Adds SVG/HTML diagrams next to text while preserving local-only and XSS-safe rendering.	New <code>core/diagram-renderer.js</code> , <code>data/fragment-meta-schema.json</code> , <code>core/ui.js</code>
P1	Replace per-entry BM25 rescoring with an inverted index and top-k accumulator.	Turns current small-KB approach into scalable retrieval as bots grow.	<code>core/bm25.js</code> , bot-pack exporter
P1	Unify policy training scripts and ship a repeatable eval report per weights file.	Prevents mode/intent regressions and makes model quality measurable.	<code>dev/scripts/train-policy.py</code> , <code>dev/scripts/train_real.py</code> , <code>dev/benchmarks/*</code>
P2	Local feedback loop in IndexedDB: thumbs, answer length preference, diagram preference, per-topic mastery.	More useful and adaptive without sending data anywhere.	New <code>core/local-profile.js</code> , <code>core/session.js</code> , policy features
P2	Document drift cleanup and single docs source of truth.	Prevents stale architecture claims and improves maintainability.	<code>docs/*.md</code> , <code>how-it-works.html</code> , <code>README.md</code> , <code>llms*.txt</code>

4. Algorithm Optimizations

4.1 Retrieval: Replace $O(N \cdot k)$ BM25 Top-K With An Inverted Index

Current `BM25Scorer.scoreTopK()` performs repeated scans to find the best unused document. That is acceptable for 38 to 77 topics, but it recomputes scores repeatedly and will become wasteful when KBs grow or when fragments become independently searchable.

CURRENT	REPLACEMENT	BENEFIT
Compute score for each doc repeatedly while selecting top K.	Build term -> posting list at pack time; score only docs touched by query terms.	For sparse queries, ranking cost becomes proportional to matching postings, not all documents.
Name/alias boosting through text repetition.	Store field weights explicitly: title 4.0, alias 3.0, summary 1.5, fragment 1.0.	Cleaner ranking, no inflated doc length side effects.
Bigram bonus checked against doc bigram set.	Index bigrams as postings with lower weight.	Consistent phrase scoring with no per-doc set allocation.

```
Proposed pack-time index:
{
  "terms": {
    "nash": [[entryId, tfTitle, tfAlias, tfBody], ...],
    "nash_equilibrium": [[entryId, tfPhrase], ...]
  },
  "idf": { "nash": 1.92, "nash_equilibrium": 2.43 },
  "docLen": [...],
  "avgDocLen": 123.4
}
```

4.2 Entity Extraction: Alias Regex To Automaton + BK-Tree

`compileAliasRegex()` precompiles regexes per entry and `extractEntities()` loops through the KB. This is workable now, but adding bots, synonyms, and multilingual variants will increase cost and ambiguity.

- Build an Aho-Corasick or FlashText-style alias automaton at pack time for exact multi-token aliases.
- Keep a small BK-tree or SymSpell table for typo correction over aliases.
- Normalize accents, punctuation, Greek letters, formula tokens, and hyphen variants in one tokenizer.
- Emit entity candidates with spans and scores: `{kbId, spanStart, spanEnd, alias, score, method}`.
- Resolve overlapping aliases with longest-span-first plus graph centrality.

4.3 Dense Retrieval: Precompute And Pack Entry/Fragment Embeddings

On first dense load, the runtime encodes the KB in batches after MiniLM loads. That is a good progressive step, but it still burns CPU on every first visit per bot. For static KBs, entry and fragment vectors should be generated offline and shipped in the bot pack.

VECTOR TYPE	CURRENT	RECOMMENDED
Entry embeddings	Computed in browser after model load.	Ship Float16 or Int8 normalized vectors per bot.
Fragment embeddings	Computed lazily through <code>embedCached()</code> .	Pack fragment vectors, then use query-only embedding at runtime.
Domain prototypes	Embedded at init if configured.	Pack domain prototype vectors with versioned model hash.
Compression	JSON + ONNX assets.	Use binary <code>.bin</code> vectors with header: magic, modelHash, dim, dtype, count.

This keeps all inference local while reducing first dense-ready time. The browser still embeds the user query locally, but does not re-embed static KB text.

4.4 Ranking Fusion: Move From Fixed Weighted Sum To RRF + Learned Ranker

The current ensemble uses dense 0.7 and sparse 0.3 plus a simple rerank. Better local ranking can be achieved with:

- Reciprocal Rank Fusion (RRF) as a robust default when sparse and dense score scales disagree.
- A tiny learning-to-rank MLP or logistic calibrator over features: dense sim, BM25, entity hit, title hit, follow-up boost, fragment freshness.
- MMR for diversity: choose high-scoring topics while penalizing near-duplicates.
- Topic graph expansion after top-1 only when confidence is high or the user asks "what else".
- Confidence thresholds calibrated per bot, not globally copied across all bot configs.

4.5 Fragment Selection: Use Metadata And Diversity Earlier

Fragment selection currently embeds and softmaxes candidate fragments with a recent-fragment penalty. Stronger selection should combine:

- Semantic match: query/fragment cosine.
- Answer budget: short mode picks one high-density fragment; long mode picks def + int + ex + app/form.
- Difficulty fit: user asks "simple" -> lower difficulty; "prove" -> formal and higher difficulty.
- Freshness: penalize exact fragment repetition and semantically similar paraphrase repetition.
- Truth/source confidence: prefer high-confidence fragments when challenged or asked for evidence.
- Diagram availability: if a concept has a visual template and the user would benefit, reserve room for it.

4.6 Query Understanding: Local Expansion Without LLMs

ReLU.chat can become more intelligent without a cloud LLM by adding deterministic local understanding layers:

- Per-bot synonym maps built from aliases and related concepts.
- Local spelling correction against alias vocabulary.
- Question operator classification: what, why, how, when, compare, example, prove, summarize, challenge.
- Formula/token recognizers for game matrices, probability notation, ML metric names, and historical date ranges.
- Graph-aware expansion: if top topic is "Nash equilibrium" and intent is example, include "Prisoner's Dilemma" only when helpful.
- Contradiction/rejection detection: "not that", "I meant X", "wrong topic", "no just X".

5. Conversation Awareness

5.1 Fix The Verified Follow-Up Bug

Observed failure: `dev/benchmarks/test-followup.js` expects `how?` -> `how` and `why?` -> `why`, but the current simple detector returns `elaborate` in three assertions. The feature extractor and MLP support distinct codes 9 and 10, so the lost nuance is in session detection precedence.

Recommended fix:

- Change `singleWordMap.how` to `how` and `singleWordMap.why` to `why`.
- Move short phrase patterns for `why` and `how` before generic `more/elaborate` patterns.
- Add test cases for punctuation variants: `how?`, `how`, `how so?`, `why?`, `why so?`.
- Confirm MLP decision path uses `followup:override-how` and `followup:override-why`.

5.2 Session Memory Should Become A Structured Local State Machine

`SessionMemory` already tracks turns, entities, fragments, ambiguity, summaries, engagement signals, and an EMA embedding. The next step is to make this state explicit and queryable by policy and renderer.

```
{
  "turn": 12,
  "activeTopic": "nash_eq",
  "topicStack": ["nash_eq", "prisoners_dilemma"],
  "lastIntent": "example",
  "userDepthPreference": "medium",
  "diagramPreference": "auto",
  "activeEntities": [{"id": "nash_eq", "weight": 0.91}],
  "recentFragments": ["nash_eq:def", "nash_eq:int"],
  "unresolvedClarification": null,
  "localFeedback": {"shortAnswers": 3, "diagramsOpened": 2}
}
```

All of this should remain in memory by default. If persistence is enabled, store it in IndexedDB behind a local-only profile switch and provide a "clear local learning" action.

5.3 Wire The EMA Summary Vector Into Real Features

The session currently maintains `getSummaryEmbedding()`, but the policy feature contract does not yet include it directly. Instead of expanding the 25-feature vector immediately, add derived scalar features first:

- `queryToSessionSim`: cosine of current query to EMA vector.
- `sessionTopicStability`: how stable recent top topics are.
- `fragmentNoveltyPressure`: fraction of candidate fragments recently used.
- `engagementDepth`: rolling measure from deepening/challenging/clarifying signals.
- `answerBudgetPreference`: local profile or inferred short/medium/long preference.

If keeping exactly 25 features is important for current weights, create policy v0.5 with versioned feature schema rather than silently changing the vector.

5.4 Clarification Should Be An Answer Mode, Not Just A Fallback

The action schema already has `mode: clarify`. Use it more aggressively when:

- Dense and sparse disagree strongly and no entity is found.
- Top two topics are close and the user query is short.
- The user uses pronouns without a stable previous topic.
- A correction target cannot be matched with high confidence.

Clarification should be concise, with 2 to 4 local options, and should update session state so the next answer resolves the pending ambiguity.

6. Answer Variation And Length Control

The current system varies answers through fragment choice, connectors, openers, closers, and policy-selected fragment counts. The user request needs a more explicit automatic length and variation system.

6.1 Add Answer Budget To AnswerPlan

BUDGET	POLICY MEANING	RENDERER CONTRACT
micro	Greeting, quick correction, direct definition.	1 fragment, no related topics, no closer unless useful.
short	User asks quick/simple, mobile, low confidence, or repeated topic.	1 to 2 fragments, max 80 words, optional tiny diagram.
medium	Default for most desktop and first educational answers.	2 to 3 fragments, max 170 words, one related topic.
long	User asks deep, formal, evidence, or "tell me more".	3 to 5 fragments, sectioned answer, example or formula.
diagram_plus_text	Concept has a strong visual representation.	Text plus safe SVG/HTML diagram AST.

6.2 Infer Budget Locally

- Lexical cues: "brief", "tl;dr", "short", "summary" -> short.
- Depth cues: "prove", "derive", "why", "how exactly", "detailed" -> long.
- Device hints: small screen, low memory, or Save-Data -> start short unless user asks deep.
- Conversation state: if user repeatedly asks follow-ups, increase detail gradually.
- Local feedback: thumbs up on short answers raises prior for short answers on that device.
- Uncertainty: low confidence should trigger clarify or short answer with "I may be reading this as..."

6.3 Deterministic Variation

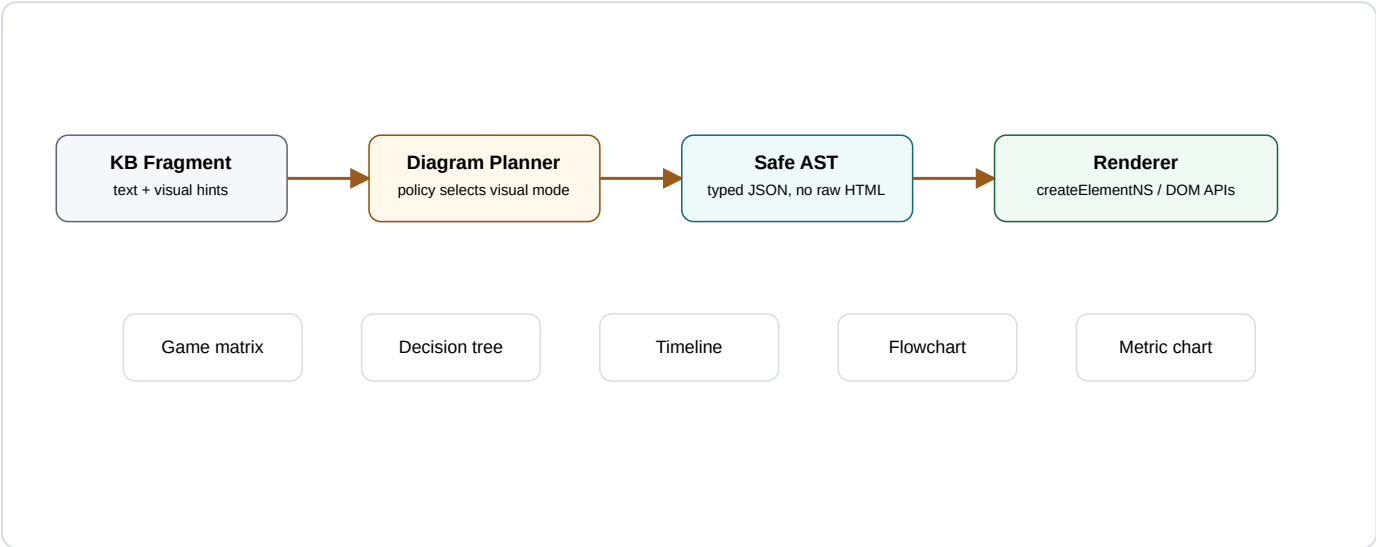
Variation should be reproducible for the same local state so tests and user experience are stable. Use a seed based on bot id, KB version, normalized query, selected topics, turn number, answer budget, and creativity bucket.

```
seed = hash32(botId + kbVersion + normalizedQuery + topics.join(",") + turn + budget + creativityBucket)
setCompositionSeed(seed)
renderer picks opener, connectors, examples, and closers from seeded RNG only
```

This also lets the user request "another version" by incrementing a local variant counter without changing the semantic plan.

7. Visual SVG/HTML Diagram Layer

The safest way to add visual answers is not to let fragments contain raw HTML. Instead, add a diagram AST schema that the renderer turns into DOM/SVG nodes. This keeps everything local, inspectable, and XSS-resistant.



7.1 Diagram Types To Add First

BOT	DIAGRAM TYPE	EXAMPLES	WHY IT HELPS
Game theory	2x2 payoff matrix, extensive-form tree, equilibrium marker, auction flow.	Prisoner's Dilemma, Chicken, Stag Hunt, Nash equilibrium, minimax.	Game theory is inherently visual; a matrix often explains faster than paragraphs.
Data science	Confusion matrix, bias-variance curve, train/test split bar, pipeline DAG, decision tree.	Precision/recall, cross-validation, leakage, PCA, ensembles.	Many ML concepts are process or matrix concepts; diagrams reduce cognitive load.
Golden Age	Timeline, scholar influence graph, invention flow, map-lite schematic.	House of Wisdom, translation movement, optics, algebra, astronomy.	Historical relationships are easier to understand with chronology and influence.

7.2 Diagram AST Example

```
{
  "type": "payoff_matrix",
  "title": "Prisoner's Dilemma",
  "players": ["Player 1", "Player 2"],
  "rows": ["Cooperate", "Defect"],
  "cols": ["Cooperate", "Defect"],
  "cells": [
    {"payoff": [3,3], "label": "mutual cooperation"},
    {"payoff": [0,5], "label": "P1 exploited"},
    {"payoff": [5,0], "label": "P2 exploited"},
    {"payoff": [1,1], "label": "Nash equilibrium", "highlight": true}
  ],
  "caption": "Defect/defect is stable even though cooperate/cooperate is better for both."
}
```

7.3 Renderer Safety Rules

- Never inject raw diagram HTML from KB strings.
- Use typed AST objects validated against a JSON schema.
- Create SVG nodes with `document.createElementNS` and text nodes with `textContent`.
- Allow only known style tokens: color roles, emphasis, highlight, muted.
- Render responsively: diagrams must fit within chat width on mobile and support horizontal scroll only for wide matrices.
- Cache rendered diagram AST by fragment id and theme.
- For accessibility, include title, caption, and text table fallback.

7.4 Policy Integration

Add new fields to `AnswerPlan`:

```
{
  "visualMode": "none | inline | card | expandable",
  "visualType": "payoff_matrix | tree | timeline | flow | chart",
  "visualRef": "fragment-or-topic-diagram-id",
  "visualPlacement": "before_text | after_intro | after_text",
  "visualBudget": "tiny | standard | detailed"
}
```

The policy should choose visuals when a diagram is available, confidence is high, and the answer budget permits it. Mobile can default to `expandable`; desktop can render inline.

8. RL And Policy Pipeline Improvements

8.1 Current Policy State

- The runtime policy is a local pure-JS MLP with 25 inputs, 128 and 64 hidden units, and six action heads.
- The shipped runtime manifest is version 0.4.0 and the active JSON weights file is about 362 KB.
- A 43-byte policy WASM exists as a stub-like artifact; the JS MLP is the real practical path today.
- Policy dev tests pass, but the 10-scenario benchmark reports only 30% intent accuracy.
- Training scripts include both a broad `train-policy.py` pipeline and a newer `train_real.py` pipeline with anti-collapse measures.

8.2 Make Training One Reproducible Pipeline

Merge the useful parts of `train-policy.py` and `train_real.py` into a single command that produces a signed artifact directory:

```
dev/exports/policy-runs/2026-06-10Txx/
  config.json
  dataset-summary.json
  train-metrics.jsonl
  validation-report.json
  confusion-matrix.json
  policy.weights.json
  policy.weights.bin
  policy.manifest.json
  eval.html
```

The manifest should include feature schema version, reward weights, dataset hash, KB hash, model hash, and evaluation scores. Runtime should refuse mismatched feature schemas.

8.3 Improve Rewards

REWARD COMPONENT	CURRENT NEED	PROPOSED SIGNAL
Intent match	Intent accuracy is weak in benchmark.	Supervised pretraining plus RL reward for exact and compatible intent.
Topic precision	Topicality matters more than stylistic variation.	Reward exact gold topic, related-topic fallback, and penalize unrelated additions.
Answer budget	No explicit short/medium/long control.	Reward budget matching based on query labels and local preference labels.
Diagram choice	No visual answer policy yet.	Reward diagram use only when a validated diagram exists and query type benefits.
Follow-up continuity	Needs strong behavior for "how", "why", "what else".	Reward continuity with prior topic when follow-up type is nonzero.
Correction handling	Topic correction is valuable.	Large reward for corrected topic top-1 and concise apology-free refocus.
Repetition penalty	Fragment repetition degrades user trust.	Use local session fragment history to penalize repeats unless user asks to repeat.
Confidence calibration	Need reliable clarify vs answer decisions.	Brier/ECE penalty on confidence bins in validation.

8.4 Add Offline Golden Tests

Create a per-bot golden set with at least these buckets:

- Direct definitions for every topic alias.
- Examples for every topic that has example fragments.
- Formal questions for formula-heavy topics.
- Application questions for practical topics.
- Comparison pairs from related topic graph.
- Short follow-ups: how, why, example, simplify, summarize, what else, prove it, compare.
- Topic corrections and rejections.
- Off-topic and ambiguous queries.
- Mobile style queries: terse, typo-heavy, punctuation-only.

A policy build should fail if mode, intent, topic, or answer budget regression exceeds a threshold. This matters more than total reward curves.

8.5 Do Not Over-Optimize The Tiny MLP First

The policy model is about 13K parameters. Optimizing the MLP into WASM is not the top performance opportunity compared with MiniLM, ONNX Runtime, KB encoding, and main-thread scheduling. Prioritize model/asset loading and

retrieval packaging first.

9. Asset Loading And Mobile/Desktop Performance

9.1 Current Load Cost

ASSET / AREA	OBSERVED SIZE	RECOMMENDATION
<code>assets/models/all-MiniLM-L6-v2/onnx/model_quantized.onnx</code>	22.97 MB	Keep local, but do not force on low-end mobile before user intent. Precompute KB vectors so only query embedding needs runtime model.
ONNX Runtime WASM variants	Four files, about 38 MB total.	Serve only the likely-needed variant for the browser, or let Transformers.js request it on demand instead of pre-caching all variants.
<code>assets/transformers/transformers.js</code>	898 KB	Lazy load only inside chat pages; keep landing page free of ML bundles.
KaTeX	1.5 MB folder plus CSS/fonts.	Load on demand when a response contains math, or split math-heavy bots from non-math bots.
Hero videos	About 2.48 MB each.	Use <code>preload="metadata"</code> or adaptive loading on mobile and Save-Data; keep poster meaningful.
Policy weights JSON	362 KB global; per-bot weights also present.	Use per-bot binary or compressed JSON; avoid loading unused per-bot variants.
Dev logs/backups	<code>dev/logs</code> 23 MB, <code>dev/_backups</code> 82 MB.	Already ignored; keep out of deploys and consider local pruning/archival.

9.2 Workerized Load Plan

The chat UI should become usable immediately and never freeze. Recommended sequence:

1. Main thread renders chat shell, suggestions, local cached history, and input.
2. Worker loads bot pack and sparse indexes.
3. Worker returns `basic_ready`: BOW/BM25 answers available.
4. Worker loads policy JSON/bin and returns `policy_ready`.
5. Worker loads MiniLM/ONNX runtime when needed and returns progress events.
6. Worker hot-swaps dense vectors and returns `dense_ready`.
7. All later query requests use worker messages with cancel tokens.

9.3 Mobile-Specific Rules

- If `navigator.connection.saveData` is true, default to sparse/BOW mode until the user asks for richer answers.
- If `navigator.deviceMemory` is low, avoid caching all fragment embeddings and diagrams at once.

- Use short answer budget by default on narrow screens unless the query asks for depth.
- Make large diagrams expandable with a text fallback and no layout shift.
- Do not preload the 22.97 MB model on the landing page; wait for chat page or explicit install/warm-up.
- Throttle background model preloads while the tab is hidden or battery saver is active.

9.4 Service Worker Improvements

- Use a manifest-driven cache list from bot packs instead of hardcoded model paths.
- Separate app shell cache, bot pack cache, model cache, and generated local user data.
- Add cache size accounting and a "clear model cache" UI action.
- Use stale-while-revalidate for code, cache-first for versioned bot packs and model files.
- Cache only one ONNX Runtime variant after feature detection.
- Include policy and bot-pack manifest hashes in cache keys to avoid stale weights.

10. Local-Only Data Model

The user requirement is clear: make the system smarter while keeping all data locally in the user's browser. That is feasible if personalization is implemented as optional local state and never syncs.

10.1 What Can Be Stored Locally

LOCAL DATA	PURPOSE	STORAGE	PRIVACY RULE
Recent sessions	Continue context after reload.	IndexedDB, capped by turns and age.	Off by default or clear opt-in; user can delete.
Local preferences	Short vs long answers, diagrams, tone.	IndexedDB/localStorage small profile.	No account identity.
Feedback events	Local reranking and budget tuning.	IndexedDB event log, capped.	Never sent; export/delete buttons.
Model/bot caches	Offline and faster repeat visits.	Cache API + IndexedDB.	Versioned, clearable.
Per-topic mastery	Adapt explanation level.	Small local map.	No external analytics.

10.2 Local Adaptation Without Retraining

- Adjust answer budget priors from local feedback.
- Boost fragments the user expanded or marked useful.
- Penalize fragments the user downvoted, only on the same device.
- Remember that the user prefers diagrams for a bot or topic.
- Prefer simpler fragments if the user often asks "simpler".
- Keep an on-device "local profile vector" of preferences, not raw sensitive transcripts.

10.3 Local-Only UX Requirements

- Settings panel: "Local learning: on/off", "Clear local chat memory", "Clear downloaded models".

- Per-chat banner: "This preference is stored only in this browser."
- Export local profile as JSON for user-controlled backup.
- Do not add telemetry endpoints or remote feedback calls.
- Tests should assert no external network URLs are used for model or feedback data.

11. Separation Of Concerns And Maintainability

11.1 Proposed Module Boundaries

LAYER	RESPONSIBILITY	SHOULD NOT KNOW ABOUT
Bot pack loader	Fetch/cache bot data, indexes, vectors, config.	DOM rendering.
Tokenizer/entity layer	Normalize text, aliases, spans, fuzzy matches.	Policy heads or CSS.
Retriever	BM25, dense ranking, fusion, MMR.	Chat DOM or welcome copy.
Session memory	Local turn state, user preferences, feedback.	Model loading details.
Policy runtime	Feature extraction and AnswerPlan generation.	Raw HTML or SVG nodes.
Composer	Text/diagram plan assembly from safe fragments.	Network/cache APIs.
Renderer	DOM, markdown subset, KaTeX, diagram AST.	Ranking algorithms.

11.2 Remove Or Quarantine Legacy Chat NLP

Current app wrappers import the shared engine, but legacy files `chat/game-theory-chat/js/nlp.js` and `chat/golden-age-inquiry/js/nlp.js` still contain old composition logic and unseeded randomness. Options:

- Delete after verifying no HTML imports them.
- Move to `dev/_quarantine` with an explanation if historical reference matters.
- Or convert them into thin re-exports from `core/nlp.js` until all references are gone.

11.3 Bot Pack Shape

```
{
  "botId": "game-theory-chat",
  "kbVersion": "1.3.0",
  "model": {"embedding": "all-MiniLM-L6-v2", "dim": 384, "vectorDtype": "float16"},
  "config": {"thresholds": "...", "composition": "..."},
  "entries": "entries.json",
  "fragments": "fragments.json",
  "aliasIndex": "aliases.bin",
  "bm25Index": "bm25.bin",
  "entryVectors": "entry-vectors.f16",
  "fragmentVectors": "fragment-vectors.f16",
  "diagrams": "diagrams.json",
  "policy": {"weights": "policy.weights.bin", "manifest": "policy.manifest.json"}
}
```

This format lets a chat page fetch one manifest, then exactly the assets needed for that bot and device.

11.4 Documentation Source Of Truth

The live HTML docs and Markdown docs are not fully aligned. For example, one Markdown doc still describes a first query as 2 to 8 seconds and 55+ game-theory concepts, while current architecture has progressive first-turn handling and the manifest reports 77 game-theory topics. Generate public docs from a single architecture data file or add a doc drift test.

12. Testing, Benchmarks, And Observability

The current test footprint is solid for helpers and schemas. The next step is behavior and performance proof.

12.1 Tests To Add

- Browser smoke test per chat page: shell renders, welcome shown once, suggestions work, first BOW answer works, dense hot-swap occurs.
- Service worker cache test: model cache, Range handling, stale cache upgrade, offline repeat visit.
- Answer budget tests: short/medium/long query labels produce expected fragment counts and word caps.
- Diagram renderer tests: AST validation rejects unsafe keys and renders accessible SVG.
- Policy golden tests: per-bot mode/intent/topic/budget/diagram decisions.
- Mobile viewport tests: input remains visible, diagrams fit, no content overlap, no giant first-paint downloads on landing page.
- Network guard test: no remote model URLs, no telemetry, no feedback uploads.

12.2 Performance Metrics To Track

METRIC	TARGET	HOW TO MEASURE
Chat shell time to interactive	< 500 ms desktop, < 1000 ms mobile	Playwright performance marks.
First basic answer	< 500 ms after submit once app shell loaded	Instrument BOW/BM25 path.
Dense ready	Background, no UI freeze	Worker progress and long-task observer.
Query embedding cache hit	> 60% for repeated/local follow-ups	Local debug counters.
Policy inference	< 5 ms	Performance marks in <code>planAnswer</code> .
Diagram render	< 16 ms for standard diagrams	Renderer microbench.
Main-thread long tasks	0 during model load and answer generation	PerformanceObserver.

12.3 Local Debug Panel

Add a dev-only debug drawer, gated by query param or localStorage flag, that shows: selected mode, intent, answer budget, visual mode, topic ids, sparse/dense scores, confidence, follow-up type, policy decision path, cache state, and timings. This helps tune behavior without sending telemetry.

13. Implementation Tracks

Track A: Conversation And Answer Control (Fastest High-Impact)

1. Fix `how? / why?` follow-up mapping and tests.
2. Add `answerBudget` to `AnswerPlan` schema with validation.
3. Add local heuristic budget classifier before retraining MLP.
4. Make renderer enforce word/fragment caps by budget.
5. Wire deterministic composition seed and remove remaining `Math.random()` from active code.
6. Add golden tests for short/medium/long and "another version".

Track B: Bot Pack And Loading Speed

1. Create a dev exporter that builds `bot.pack.json` plus vectors/indexes for one bot.
2. Load bot pack from `core/chatbot-engine.js` while keeping old shape as fallback.
3. Precompute entry and fragment vectors; stop embedding static KB in browser.
4. Add feature-detected ONNX Runtime variant selection.
5. Move model loading and ranking to a Worker.
6. Update service worker to cache manifest-driven assets only.

Track C: Visual Diagrams

1. Define diagram JSON schema and renderer for payoff matrix, confusion matrix, timeline, flowchart.
2. Add 10 diagram ASTs manually to high-value topics.
3. Extend fragment metadata with `visual_refs` and `visual_fit`.
4. Add `visualMode` and `visualRef` to policy plan.
5. Test mobile rendering and accessibility text fallback.
6. Expand with generator scripts once schema is stable.

Track D: Policy And RL Quality

1. Unify training scripts and artifact manifests.
2. Add per-bot golden evaluation with confusion matrix.
3. Label answer budget and diagram use in training samples.
4. Train policy v0.5 with feature schema versioning.
5. Fail CI if mode/intent/topic/budget regress beyond thresholds.
6. Use local shadow compare to collect dev-only mismatch logs, not production telemetry.

14. Detailed Recommendation Backlog

#	RECOMMENDATION	CATEGORY	LOCAL-FIRST CONSTRAINT
1	Fix <code>how</code> / <code>why</code> follow-up mapping.	Conversation	No storage/network needed.
2	Add answer budget to <code>AnswerPlan</code> .	Policy	Decision remains local.
3	Implement deterministic seeded composition for all active render paths.	Renderer	Seed derived from local state only.
4	Remove or quarantine legacy per-chat NLP files.	Maintenance	No runtime data impact.
5	Export precomputed entry vectors.	Loading	Static local model artifacts.
6	Export precomputed fragment vectors.	Loading	Static local model artifacts.
7	Use Float16/Int8 vector packs instead of JSON arrays.	Loading	Served as same-origin static assets.
8	Build a BM25 inverted index in bot pack.	Retrieval	No server search.
9	Replace alias regex loop with alias automaton.	Entity	Index ships in pack.
10	Add BK-tree typo correction for aliases.	Entity	Runs in browser memory.
11	Add reciprocal rank fusion as a retrieval mode.	Retrieval	Uses local sparse/dense scores.
12	Add MMR topic diversification.	Retrieval	Uses local vectors and graph.
13	Train tiny local learning-to-rank head.	Retrieval	Weights ship statically.
14	Expose confidence bins and calibration metrics.	Quality	Dev-only local metrics.
15	Persist optional local profile in IndexedDB.	Personalization	No sync; clear/export controls.
16	Add local feedback controls per answer.	Personalization	Events stay on device.
17	Build diagram AST schema.	Visual	No raw remote HTML.
18	Add payoff matrix SVG renderer.	Visual	DOM/SVG only.
19	Add confusion matrix SVG renderer.	Visual	DOM/SVG only.
20	Add timeline renderer for historical bot.	Visual	DOM/SVG only.
21	Lazy-load KaTeX only when needed.	Loading	Same local files.
22	Stop preloading full ONNX on Save-Data/mobile until needed.	Mobile	No network beyond static model when requested.
23	Move inference/ranking to Web Worker.	Performance	Worker uses same-origin assets.
24	Add cancel tokens for stale query work.	Performance	Local worker protocol.
25	Add Playwright mobile/desktop smoke tests.	Testing	Local browser tests.

26	Add no-remote-model network guard test.	Privacy	Asserts local-only model path.
27	Make docs generated from architecture metadata.	Docs	No runtime effect.
28	Add per-bot threshold tuning exports.	Policy	Static config.
29	Use binary policy weights in runtime.	Loading	Same-origin static file.
30	Prune or archive oversized dev logs locally.	Maintenance	Keep ignored and never deploy.
31	Add local debug panel for decision packets.	Observability	Dev-only, no telemetry.
32	Train answer budget and visual selection heads.	RL	Weights static/local.
33	Add topic mastery features from local profile.	Personalization	Opt-in local profile.
34	Add "another version" variant counter.	UX	Local deterministic seed increment.
35	Add source/fragment citations in answers.	Usefulness	Cites local KB fragments.
36	Add "show why this answer" explanation in dev mode.	Trust	Local policy trace.
37	Add offline install warm-up flow.	PWA	User-controlled cache fill.
38	Add cache quota and model clear controls.	PWA	Local cache only.
39	Split landing assets from chat assets strictly.	Loading	No change to privacy.
40	Use responsive diagram placement by viewport.	Mobile	Local render only.

15. Completion Checklist For This Report

REQUIREMENT FROM USER	EVIDENCE IN THIS DELIVERABLE
Inspect everything about relu.chat.	Repo structure, docs, dev notes, core engine, policy runtime, session, BM25, UI, service worker, chat wrappers, configs, KBs, assets, training scripts, tests, and git ignore state were inspected.
Create a PDF under <code>dev/</code> .	This HTML source is under <code>dev/</code> and is intended to be rendered to <code>dev/relu-chat-local-first-optimization-report-2026-06-10.pdf</code> .
List tons of details that would make ReLU.chat more useful.	Backlog includes answer budgets, local feedback, citations, diagrams, topic mastery, better clarification, and "another version".
Make it faster/loading much faster.	Asset sizes, bot packs, precomputed vectors, workerization, service worker cache strategy, mobile load policy.
Make it more intelligent and conversation-aware.	Follow-up bug, session state machine, EMA features, clarification, local profile, ranking improvements.
Keep all data locally in user's browser.	IndexedDB/Cache API local-only model, no telemetry, clear/export controls, no remote feedback.
Cover desktop and mobile devices.	Separate mobile rules, Save-Data, device memory, responsive diagrams, desktop inline diagrams.
Answer in variations and lengths automatically.	Explicit <code>answerBudget</code> plan, deterministic variation, budget inference rules.
Answer with visual SVG/HTML diagrams alongside text.	Safe diagram AST, render pipeline, diagram types per bot, schema and policy fields.
Focus on algorithm optimizations.	Inverted BM25, alias automaton, BK-tree, packed embeddings, RRF, learned ranker, MMR.
Focus on RL pipeline optimizations.	Unified training pipeline, reward improvements, golden eval, artifact manifests, feature schema versioning.
Focus on asset loading and separation of concerns.	Bot packs, worker protocol, service worker plan, module boundaries, legacy cleanup.
Dev-only, do not push PDF to prod or GitHub.	Root <code>.gitignore</code> ignores <code>dev/</code> ; no deploy or git push is part of this work.